CAPSTONE PROJECT REPORT

Title: Feature Engineering for Hotel Booking Cancellation Prediction

Student Name: Sai Spoorthy Eturu

Roll No: 2025eb1100176

Project Resources:

- **Dataset:** Hotel Booking Demand Dataset (Kaggle)
  https://www.kaggle.com/datasets/maryamahmadizadeh/hotelbooking
- **GitHub Repository:** FeatureEngineering-Capstone
  https://github.com/ESpoorthy/FeatureEngineering-Capstone
- **Colab Notebook:** Project Source Code
  https://colab.research.google.com/drive/190rqFlKbE26U3pUkcQRSoN4qrcvTRoSM?usp=sharing

---

EXECUTIVE SUMMARY

This capstone project evaluates the impact of comprehensive feature engineering on predicting hotel booking cancellations (`is_canceled`). The goal was not merely to build a complex model, but to prove that intelligent preprocessing and feature creation drive model performance, stability, and interpretability.

What mattered most? Dealing with extreme outliers in financial data (Average Daily Rate) and heavily skewed temporal data (Lead Time) proved to be the most critical foundational step. Without robust scaling and power transformations, distance-based models and linear classifiers failed to generalize.

What changed the performance ceiling? Feature construction—specifically interaction terms and ratios—broke the performance ceiling. Moving beyond raw data to behavioral combinations like `price_per_person` and `interaction_adr_lead_time` allowed the model to capture the actual financial risk and psychological commitment of the guest, elevating the ROC-AUC from a baseline of 0.7008 to approximately 0.7715.

Which features are most business-actionable? `Lead_time`, `special_requests_rate`, and `deposit_type` are highly actionable. Hotels can actively use these insights to enforce stricter non-refundable deposit policies for high-lead-time bookings, or dynamically offer discounts and upgrades to guests who make special requests, as they show higher engagement and a dramatically lower propensity to cancel.

TASK 1: BASELINE MODEL & "WHAT IS A FEATURE?"

A baseline Logistic Regression model was trained using minimal preprocessing (simple mean imputation and basic one-hot encoding) to predict `is_canceled`.

Metric	Score
Accuracy	0.6755
ROC-AUC	0.7008

Baseline Results

Confusion Matrix:

	Predicted: Not Canceled	Predicted: Canceled
Actual: Not Canceled	1107	149
Actual: Canceled	500	244

Interpretation: The baseline model struggles significantly with recall for the minority class, generating 500 False Negatives. It lacks the discriminatory power needed for a reliable business tool.

What is a Feature?

A feature is an individual, measurable property or characteristic of a phenomenon being observed, serving as an independent variable (column) that a machine learning model uses to predict a target outcome. Good features are highly discriminatory, independent, and generalize well across unseen data. For example, in this dataset, `lead_time` (days between booking and arrival) is an exceptionally **good feature** because it directly correlates with customer commitment and cancellation probability. Conversely, a **bad feature** would be a highly cardinal, noisy identifier like a unique `booking_ID` or a raw string like `reservation_status_date` before temporal parsing, as these cause overfitting and hold no mathematical patterns the model can learn from.

TASK 2: CURSE OF DIMENSIONALITY DEMO

To intuitively demonstrate the Curse of Dimensionality, synthetic datasets were generated using `scikit-learn`'s `make_classification` across 2, 10, 50, and 200 features. The Euclidean distance between random pairs of points was calculated and plotted in the accompanying Colab notebook.

Observations and Implications: Upon plotting the distance distributions, a distinct pattern emerges: in low dimensions (2 or 10), there is a wide variance in distances—some points are clearly close neighbors, while others are far apart. However, as dimensions increase to 50 and 200, the distribution of distances narrows dramatically and shifts to the right. The ratio of the distance to the nearest neighbor versus the farthest neighbor approaches 1. High dimensionality makes learning exponentially harder because the very concept of ”proximity” loses its meaning; all data points become effectively equidistant from one another in the vast, sparse hypervolume. For algorithms relying on distance (like KNN or clustering) or margin-based boundaries, this sparsity destroys discriminatory power. This perfectly illustrates the necessity of feature engineering and selection: we must intentionally reduce the feature space to retain only the most informative dimensions, thereby dense-ifying the data and restoring meaningful geometric relationships for the model to learn from.

TASK 3: NUMERIC PREPROCESSING

Six numeric columns were selected for deep preprocessing: `lead_time`, `adr` (Average Daily Rate), `stays_in_weekend_nights`, `stays_in_week_nights`, `previous_cancellations`, and `booking_changes`.

- **Binning (2 features):**
  - `lead_time`: Transformed using *quantile bins* (e.g., Q1: "Last Minute", Q4: "Early Bird") to handle the heavy right skew.
  - `adr`: Transformed using *equal-width bins* to categorize bookings into distinct budget tiers.
- **Binarization (1 feature):** `high_value_customer` was created, outputting 1 if the `adr` exceeds the 85th percentile threshold, and 0 otherwise.

Scaling Comparison

The distributions and summary statistics (Mean, Std, IQR) were analyzed before and after applying three scalers:

1. **MinMaxScaler:** Forced data into a [0, 1] range. However, massive `adr` outliers ($> \$5000$) crushed 99% of the normal data into a tiny range near 0, destroying variance.
2. **StandardScaler:** Centered the mean to 0 and standard deviation to 1. Still heavily influenced by extreme values in `lead_time`, resulting in skewed summary stats.
3. **RobustScaler:** Scaled using the median and IQR. The IQR remained perfectly preserved relative to the median, and extreme outliers were isolated without distorting the majority distribution.

Conclusion: RobustScaler is definitively the best for this dataset. Features like `adr` and `lead_time` contain extreme, valid outliers. RobustScaler protects the underlying distribution of the normal customer base while keeping outliers mathematically intact but isolated.

TASK 4: DISTANCE / PROXIMITY METRICS & IMPACT

A K-Nearest Neighbors (KNN) classifier was utilized to test the impact of spatial scaling and distance metrics.

KNN Performance Metrics

Scaling Method	Distance Metric	Accuracy Score	ROC-AUC
No Scaling	Euclidean	0.6512	0.6120
StandardScaler	Euclidean	0.7645	0.7410
RobustScaler	Euclidean	0.7780	0.7555
RobustScaler	Manhattan	**0.7812**	**0.7602**

Observations on Results and Outlier Sensitivity:

1. **Why scaling changes learning:** Without scaling, features with large numerical magnitudes (like `lead_time` ranging up to 700) completely dominate the Euclidean calculation over crucial features with small ranges (like `special_requests` ranging 0-5). Scaling equalizes the geometrical weight of all features.
2. **Outlier Sensitivity (Observation 1):** StandardScaler's performance was degraded slightly by outliers pulling the mean. RobustScaler provided a tighter decision boundary because the core data clusters were standardized symmetrically via the IQR.
3. **Metric Sensitivity (Observation 2):** Manhattan distance (L1 norm) outperformed Euclidean (L2 norm). Because Euclidean squares the differences, it penalizes outliers exponentially. Manhattan distance sums absolute differences, making it far more robust and less sensitive to the extreme anomalies present in the hotel dataset's high-dimensional space.

TASK 5: END-TO-END NUMERIC PIPELINE

A modular `scikit-learn` Pipeline was constructed to automate preprocessing and ensure reproducibility without manual copy-pasting. The pipeline utilizes a `ColumnTransformer` to apply specific logic to distinct feature types.
Pipeline Configuration:

- **Imputation:** `SimpleImputer(strategy='median')` for robust handling of missing numericals.
- **Transformation:** `PowerTransformer(method='yeo-johnson')` applied strictly to highly skewed features like `adr` and `lead_time` to stabilize variance and create Gaussian-like distributions (handling zero values better than standard `np.log1p`).
- **Scaling:** `RobustScaler()` applied globally to the output.
- **Model:** Logistic Regression.

Cross-Validation Score (5-Fold): The pipeline achieved a highly stable CV mean ROC-AUC of **0.7680**. Encapsulating these steps within a Pipeline is critical as it guarantees that imputation parameters (like median) and scaling metrics are learned strictly from the training folds, preventing data leakage into the validation folds.

TASK 6: FEATURE EXTRACTION

Features were extracted across multiple data types to expose hidden predictive signals.

1. **Date/Time:** `arrival_month` extracted from the arrival date to capture seasonal peaks in cancellations (e.g., high summer volume).
2. **Date/Time:** `is_weekend` (binary) indicates if the stay includes Saturday/Sunday, differentiating leisure guests (lower cancellation risk) from corporate travelers.
3. **Text/Categorical:** `country_name_encoded`. Instead of standard text processing, high-cardinality country codes were grouped into synthetic regional text labels (e.g., "Western_Europe", "Domestic_PT"), capturing macroeconomic travel friction.

4. **Categorical (OHE):** `market_segment` processed via `OneHotEncoder` to explicitly map the different cancellation behaviors of Direct vs. Corporate vs. Online Travel Agents (OTA).

5. **Categorical (Target):** `deposit_type` processed via *Target Encoding*. (Note on leakage: To prevent data leakage, target encoding was calculated inside the cross-validation fold using 'category_encoders', ensuring test data targets did not bleed into the mapping probabilities).

TASK 7: FEATURE CONSTRUCTION

To provide the model with deeper contextual interactions, 8 features were mathematically constructed using domain logic.

1. **Ratio 1 - `price_per_person`:** `adr / (adults + children + 1)`. Standardizes financial burden. High per-person costs heavily correlate with shopping around and canceling.

2. **Ratio 2 - `special_requests_rate`:** `total_of_special_requests / (total_nights + 1)`. Captures guest engagement per day of stay.

3. **Interaction 1 - `adr_lead_time_risk`:** `adr * lead_time`. Represents total financial risk over the waiting period. A high value signifies massive commitment friction.

4. **Interaction 2 - `total_guests_weekend`:** `(adults + children) * is_weekend`. Captures the dynamic of large families booking leisure weekend stays.

5. **Aggregated 1 - `avg_adr_by_hotel`:** Grouped average pricing mapped back to the row, giving a baseline to compare if the specific booking is over or under-priced.

6. **Aggregated 2 - `avg_cancellations_by_agent`:** Historical reliability of the booking agent.

7. **Polynomial 1 - `lead_time^2`:** Generated via `PolynomialFeatures(degree=2)`, amplifying the exponential drop-off in reliability for bookings made months in advance.

8. **Polynomial 2 - `adr^2`:** Captures the non-linear relationship between extreme luxury pricing and sudden cancellations.

Avoiding Leakage in Feature Construction

- **Risk 1 - Target-Based Aggregation Leakage:** Grouping by Agent and calculating the `mean(is_canceled)` using the whole dataset leaks test labels. *Prevention:* Aggregated features were constructed strictly using statistics derived *only* from the X_train subset post train-test split.

- **Risk 2 - Time-Series Look-ahead Leakage:** Creating a ratio using features determined at checkout (like `reservation_status_date`) to predict a pre-arrival cancellation. *Prevention:* Excluded any variables updated post-booking-confirmation from the construction formulas.

- **Risk 3 - Global Imputation Leakage:** Filling zero-values in `adr` with the global median before creating the `price_per_person` ratio. *Prevention:* Pushed all feature construction into custom `FunctionTransformer` steps inside the scikit-learn pipeline, ensuring they act only on properly isolated fold data.

TASK 8: FEATURE IMPORTANCE + SELECTION

Three distinct methodologies were used to evaluate feature importance and filter out redundant dimensions.

Part A: Feature Importance Methods

- **RandomForest Importance (Tree Impurity):** Heavily favored continuous interaction variables. (Top: `lead_time`, `adr_lead_time_risk`, `deposit_type_encoded`, `price_per_person`, `country_encoded`)
- **Mutual Information (`mutual_info_classif`):** Measured universal statistical dependence. (Top: `deposit_type_encoded`, `lead_time`, `previous_cancellations`, `adr_lead_time_risk`, `special_requests_rat`
- **Permutation Importance (Bonus):** Evaluated feature impact on unseen validation data by shuffling columns. Confirmed that removing `deposit_type` and `lead_time` caused the largest drops in AUC.

Part B: Filter Feature Selection

- **Correlation Filtering:** Removed highly collinear features. `adr` and `price_per_person` had a Pearson correlation > 0.88; `adr` was dropped in favor of the ratio. `stays_in_week_nights` was dropped due to high correlation with constructed `total_nights`.
- **Chi-Square:** Applied to positive-only categorical/binned features to validate relationships.

Overlaps, Disagreements, and Final Selection: Both RF and MI agreed perfectly that `deposit_type`, `lead_time`, and the constructed `adr_lead_time_risk` were paramount. RF slightly overrated continuous polynomial features due to high cardinality, while MI correctly identified the importance of binary indicators like `special_requests`.

Final Selection: A final feature set of exactly **Top 20 features** was selected. This discarded 60% of the original/OHE dimensions (like obscure market segments) that provided zero information gain. Justification: This tight selection removes noise, entirely bypasses the curse of dimensionality, and creates a highly stable, computationally efficient model for deployment.

FINAL TASK: BEFORE VS AFTER FEATURE ENGINEERING

Version	Features	Preprocessing	Model	ROC-AUC	Notes
Baseline	31	Simple Imputation, OHE	LogReg	0.7008	High false negatives; poor detection of complex risks.
After Preprocessing	31	Yeo-Johnson, RobustScaler	LogReg	0.7410	Massive jump due to stabilizing adr/lead time outliers.
After Extraction	45	Target Encodings, Interactions	LogReg	0.7780	Domain-logic ratios (price/person) capture human behavior.
After Selection	20	Correlation filter, MI Top 20	LogReg	0.7715	Minimal performance drop, but model is 2x faster & stable.